Calidad de Software A

TP Evaluativo

Federico Calla Aliende - Salvador Munduteguy - Francisco Marino

Versión 1.0

TESTING DE CLASE AsociadoDTO

Se nos pide desarrollar una “Suite de pruebas unitarias” para la clase AsociadoDTO que compruebe el correcto funcionamiento de la asignación, recuperación y las validaciones internas de seguridad implementadas en los setters del objeto.

Analizando la especificación de requisitos específicos de la clase AsociadoDTO encontramos la siguiente información;

- La clase funciona como contenedor de datos.
- Los atributos de la clase junto con sus tipos de datos y sus reglas de validación.

Atributo	Tipo de Dato	Reglas de Validación
nombre	String	No puede ser nulo
apellido	String	No puede ser nulo
dni	String	No puede ser nulo Actúa como identificador único
ciudad/calle	String	No pueden ser nulos
numero	int	Debe ser mayor o igual a 0
teléfono	String	No puede ser nulo

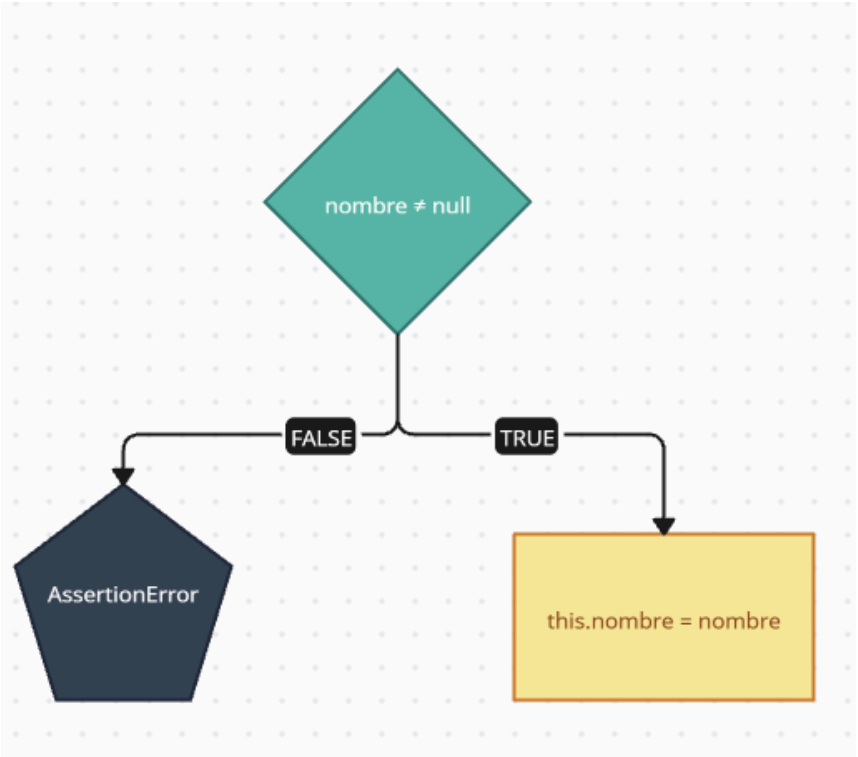
Con esta información podemos definir las clases de equivalencia y los valores límite.

Atributo	Clase Válida	Clase Inválida	Valores límite
nombre	nombre != null	nombre == null	nombre == null nombre == “Pedro”
apellido	apellido != null	apellido == null	apellido == null apellido == “Perez”
dni	dni != null	dni == null	dni == null dni == 12345678
ciudad/calle	ciudad != null	ciudad == null	ciudad == null ciudad == “Mar del Plata”
numero	numero >= 0	numero < 0	numero == -1 numero == 0
telefono	telefono != null	telefono == null	telefono == null telefono == “223 457 9863”

Nótese que en varias ocasiones podrían encontrarse valores que semánticamente no son correctos. Como por ejemplo, en el caso del dni, se sabe que ningún dni está compuesto por letras, o al hablar del nombre podríamos ingresar el valor “” ya que el SRS nos indica que no se permite que el campo nombre sea nulo, pero no especifica nada acerca de la cadena nula. Sin embargo, estos casos no forman parte de los valores límites pues no son limitaciones impuestas por la especificación de requisitos de la clase por lo que no serán tenidos en cuenta a la hora de realizar las pruebas.

Setters

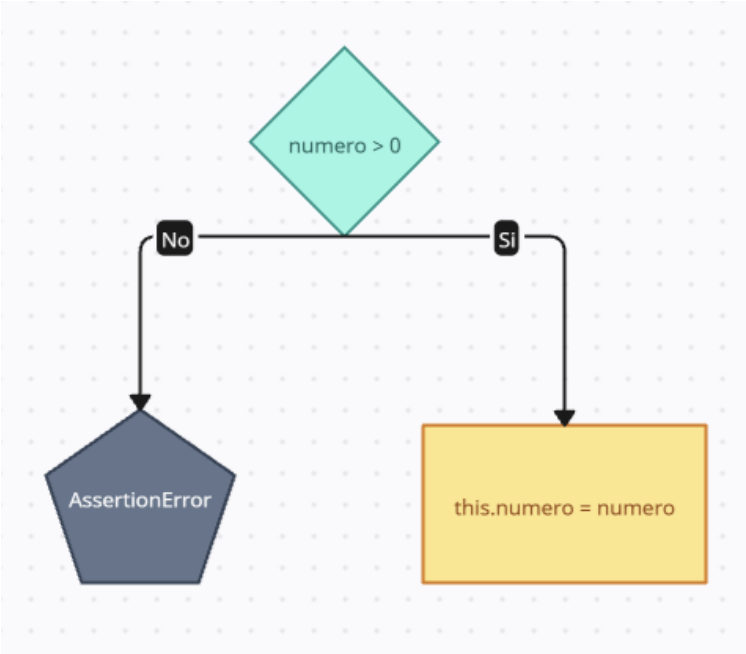
El método setNombre() contiene un assert que determina si el nombre es nulo o no. La cantidad de caminos posibles para tomar son 2.



Si se intenta asignar el valor *null* a this.nombre deberíamos obtener un AssertionError. En caso de que nombre no sea *null*, debería asignarse el valor al asociado.

Este grafo puede asociarse a los setters de nombre, apellido, dni, ciudad, calle, y teléfono dado que los métodos tienen la misma estructura, por lo que representar gráficamente cada uno de ellos sería redundante.

En el caso de **numero**, el método setNumero() contiene una estructura similar pero en lugar de consultar si número es *null*, consulta si número es mayor a 0.

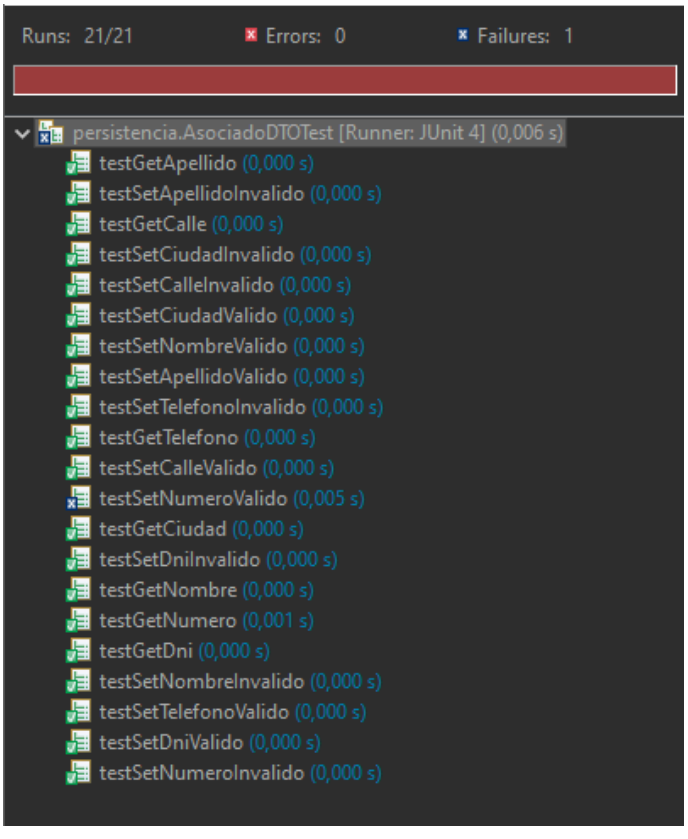


Diseñaremos ahora los casos de prueba de los setters con la información previamente obtenida.

Casos de Prueba DTO

Ejecución de prueba unitaria sobre AsociadoDTO

Una vez construida la suite de pruebas para la clase AsociadoDTO, procederemos a ejecutarlas.



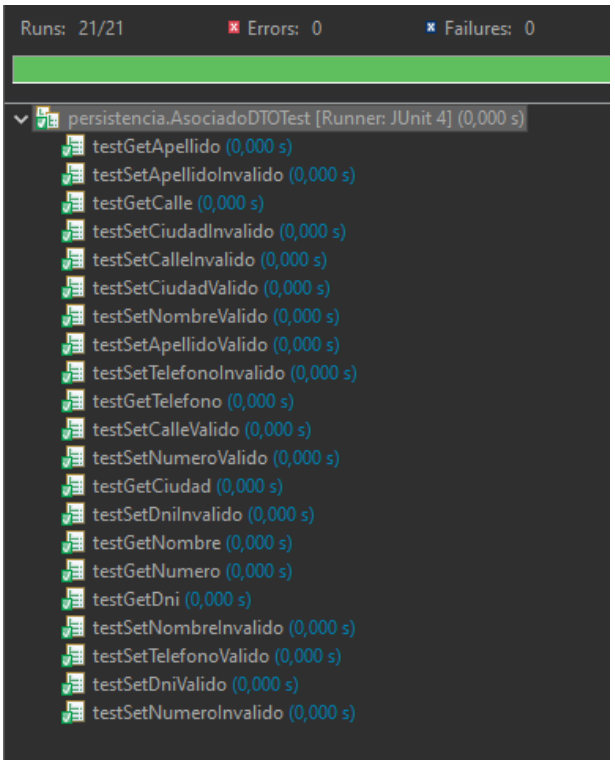
Al ejecutar las pruebas encontramos un **error** en el test de setNumero().

Al analizar el stack trace completo del error encontrado en testSetNumeroValido(), encontramos:

java.lang.AssertionError at AsociadoDTo.java: 87

Esto corresponde a la línea: *assert numero > 0* del método setNumero() de AsociadoDTO.

La suite de pruebas entonces, fue capaz de encontrar una inconsistencia entre el código y la especificación de requerimientos (Donde se especificó que 0 era un valor válido para *número*). Procederemos a cambiar el código de setNumero() para que cumpla con lo solicitado en el documento de SRS.



Al ejecutar la suite de pruebas por segunda vez, habiendo realizado el cambio correspondiente, vemos que ya no existen errores ni inconsistencias por lo que podemos asegurar que AsociadoDTO funciona **correctamente**.

TESTING DE INTEGRACIÓN

Para la segunda parte del trabajo se nos solicita realizar las pruebas de los módulos AsociadoDTO, DataAccessObject y Base de Datos. Verificar la correcta ejecución del patrón Singleton en la conexión a la base de datos, la comunicación a la base de datos real MySQL, la inicialización automática de las tablas junto con algunos datos, y las operaciones CRUD.

A continuación definiremos un plan por pasos para llevar a cabo las consignas dadas.

- Verificar la correcta ejecución del patrón Singleton
- Verificar la inicialización automática de tablas y datos semilla
- Comprobar funcionamiento de operaciones CRUD con parámetros

Patron Singleton

El SRS (sección 3.2) establece que el componente **BaseDeDatos** "aplicará el patrón de diseño Singleton para restringir la instanciación a un único manejador de conexión activo durante toda la ejecución". El patrón se implementa con tres elementos:

- Un atributo estático *Instancia*
- Un constructor privado, que abre la conexión JDBC y ejecuta iniciarBD()
- El método de acceso getInstancia() que crea la instancia solo si instancia == null.

Se trata de una **prueba de integración**, ya que la creación de la instancia involucra al motor MySQL real (carga del driver, apertura de la conexión e inicialización del esquema).

Inicialmente se plantearon tres escenarios para getInstancia():

1. Cuando nunca se creó una instancia.
2. Cuando ya se creó una instancia.
3. Cuando se creó una instancia y luego se eliminó.

Escenario	Decisión	Fundamento

1. Primera invocación	Cubierto	La primera llamada a getInstancia() toma la rama instancia == null y construye el objeto.
2. Invocaciones sucesivas	Cubierto	Las llamadas posteriores toman la rama instancia != null y devuelven el mismo objeto.
3. Instancia eliminada	No testeable	La clase no ofrece ninguna operación pública para destruir o reiniciar la instancia, y ese es justamente el propósito del patrón. No existe forma de provocar ese estado desde el exterior.

Los escenarios 1 y 2 corresponden a las dos salidas de la única decisión de getInstancia(), por lo que juntos alcanzan una cobertura de decisiones del 100% sobre ese método.

ID	Descripción	Verificación	Resultado esperado
T15	Dos solicitudes de instancia devuelven el mismo objeto	assertSame(baseDeDatos, BaseDeDatos.getInstancia())	Ambas referencias apuntan al mismo objeto
T16	La instancia posee una conexión JDBC establecida	- assertNotNull("Debe existir un objeto Connection inicializado", conexión); - assertFalse("La conexión a la base de datos debe estar abierta", conexion.isClosed());	La conexión no es nula
T17	Más de dos solicitudes de instancia devuelven el mismo objeto	assertSame("La instancia en la iteración " + i + " debe ser la misma", referenciaInicial, actual);	Todas las referencias apuntan al mismo objeto

El SRS habla de un único *manejador de conexión*, mientras que el código aplica el patrón a la clase BaseDeDatos. Ambas formulaciones son equivalentes en esta implementación, porque la instancia única conserva una única Connection como atributo y todas las operaciones (ejecutarConsulta, ejecutarInsert, ejecutarActualizacion, ejecutarDelete, iniciarBD) la reutilizan. Se verificó por inspección del código.

La prueba “T15” para comprobar si dos solicitudes de instancia devuelven el mismo objeto se implementó en el método “test01_Singleton_DevuelveMismaReferencia”.

“baseDeDatos” es un atributo static de la clase BaseDeDatosTest.java, el cual es inicializado en el metodo set”UpClass()” por primera vez y antes de cada método “@Test” de con el método “setUp()” si es que no se haya inicializado.
Por otro lado, BaseDeDatos.getInstancia() devuelve el valor actual del atributo estático “instancia” de la clase BaseDeDatos.
Si el patrón singleton no estuviera bien implementado, las referencias podrían diferir.

La prueba T16 está implementada en el método “test03_Singleton_ConexionInicializadaYActiva”, el cual verifica que el método de la clase BaseDeDatos “getConexion()” devuelva la referencia a un objeto no null de tipo Connection. También el método de testeo verifica que dicha conexión no está cerrada a partir del método “conexion.isClosed()”.

Por último la prueba T17 está implementada en el método “test02_Singleton_MultiplesLlamadasNoGeneranNuevasInstancias()”. El metodo verifica que la solidez de la implementación del patrón de diseño Singleton más allá de 2 instancias, iterando un total de 5 veces la llamada al método “getInstancia()” y comprobando que cada una de las referencias apunte al mismo objeto de la clase BaseDeDatos.

Inicialización automática de tablas y datos semilla

El SRS (sección 3.2) establece que, al inicializarse, el sistema ejecutará automáticamente la rutina iniciarBD para:

1. verificar o crear el esquema de base de datos objetivo,
2. eliminar y recrear la tabla Asociados,
3. poblarla con registros iniciales de prueba.

iniciarBD() es invocado desde el constructor privado de BaseDeVerificacionVerificacionDatos, por lo que se ejecuta una sola vez, en la primera llamada a getInstancia(). También es público, así que puede invocarse de nuevo. Ejecuta cinco sentencias SQL en orden:

Paso	Sentencia	Descripción
1	CREATE DATABASE IF NOT EXISTS Grupo_7 (utf8, utf8_unicode_ci)	Crea el esquema si no existe
2	USE Grupo_7	Selecciona el esquema
3	DROP TABLE IF EXISTS Asociados	Elimina la tabla previa
4	CREATE TABLE IF NOT EXISTS Asociados (...)	Crea la tabla con id (PK autoincremental), nombre, apellido, DNI (UNIQUE), calle, número, ciudad y teléfono
5	INSERT INTO Asociados ... VALUES (...)	Carga 20 registros semilla (ids 1 a 20)

iniciarBD() no recibe parámetros ni devuelve valores, así que no se puede probar con un valor de retorno así que verificaremos sus efectos sobre la base de datos.
De todas formas utilizaremos el método ejecutarConsulta() del objeto baseDeDatos, porque incluso aunque dicho método no haya sido testeado cualquier error encontrado dentro de los método de testeo de la inicialización automática de tablas y datos semilla que coincidan con errores en la sección de métodos de consulta, serán evaluados como errores de consulta. Dejando en claro las circunstancias del testeo continuaremos con la declaración de casos de prueba para el testeo de la inicialización automática de tablas y datos semilla.

Como se mencionó anteriormente, la inicialización de la base de datos, tablas, y datos semilla da origen en el método iniciarBD() y realiza la siguientes operaciones:

1. Crear el esquema de la base de datos si no existe
2. Eliminar las tablas existentes si ya existían
3. Crear la tabla de Asociados
4. Cargar 20 datos semilla

Para estas tareas diseñamos los siguientes casos de prueba considerando los mismo escenarios que para el testeo del patrón Singleton, ya que fundamentalmente iniciadBD() se implementa dentro del getInstancia().

Tabla Casos de prueba Singleton

Una vez concluida la especificación de testing del módulo de creación de base de datos y submódulos involucrados, continuamos con el testeo de las operaciones CRUD.

Operaciones Crud

Para poder diseñar una metodología de testeo orientado al crud en el objeto BaseDeDatos, tenemos que entender cuál es la responsabilidad real del objeto en el sistema.
El srs sección 3.2 especifica que “El sistema proveera interfaces de ejecución con parámetros para consultas, inserciones, actualizaciones y eliminaciones, utilizando declaraciones preparadas **PreparedStatement** para prevenir vulnerabilidades de inyección SQL.
Con esta especificación, podemos interpretar la responsabilidad, no realmente como el crud real de la capa de persistencia del sistema, sino como el mero objeto encargado de ejecutar consultas SQL en la base de datos. De esta manera procederemos a armar el contexto y los casos de prueba para testear la funcionalidad de ejecución de consultas SQL.

Revisando el código podemos encontrar los siguientes métodos encargados de ejecutar consultas sql distinguidos por operación.

- public ResultSet **ejecutarConsulta**(String consulta, int id) throws SQLException
- public void **ejecutarActualizacion**(String actualización, AsociadoDTO aPTO) throws SQLException
- public void **ejecutarInsert**(String insert, Asociado DTO aDTO)throws SQLException
- public void **ejecutarDelete**(String delete, String dni) throws SQLException

Estos son los primeros métodos con los que podemos empezar a “jugar” con los parámetros de entrada, testeando valores válidos, inválidos y límite para los mismos escenarios.
Los contratos de los 4 métodos, tienen la misma estructura. Como precondición una consulta SQL válida, es decir, asumimos sin errores de sintaxis. Como postcondición “El resultado de la consulta” lo cual aunque a priori pareciera una postcondición válida, tal vez, sería más claro una postcondición, que explique la estructura de datos que recibimos, la

forma de acceder a los datos, si están ordenados o no, etc. Y finalmente una excepción del tipo SQLException si ocurre un error al ejecutar la consulta.

Para proceder con el testeo, entendemos que solamente podemos plantear un escenario en el que la base de datos ya está instanciada, las tablas y base de datos creada, la conexión disponible y los datos semilla correctamente insertados.

Escenario	Decisión	Fundamento
1. BaseDeDatos recién instanciada	Cubierto	La creación de la base de datos mediante a la cual accedemos a los métodos de interfaz para ejecutar querys SQL necesita haber sido instanciada y en consecuencia, ya creada las tablas, base de datos, datos y conexión.

Por último distinguimos 5 querys SQL que servirán como consultas válidas para el testeo de cada uno de los métodos.

1. **SQL_SELECT_TODOS** = "SELECT * FROM Asociados;"
2. **SQL_SELECT_POR_ID** = "SELECT * FROM Asociados WHERE id = ?;"
3. **SQL_INSERT** = "INSERT INTO Asociados (nombre, apellido, DNI, calle, número, ciudad, teléfono) VALUES (?, ?, ?, ?, ?, ?, ?);"
4. **SQL_DELETE_POR_DNI** = "DELETE FROM Asociados WHERE DNI = ?;"
5. **SQL_UPDATE** = "UPDATE Asociados SET nombre = ?, apellido = ?, DNI = ?, calle = ?, número = ?, ciudad = ?, teléfono = ? WHERE id = ?;";

ejecutarConsulta-Test → Tabla Casos de prueba Consulta

Comenzando las pruebas de los métodos de acceso a la base de datos, procederemos a plantear los casos de prueba teniendo en cuenta los contratos, parámetros y resultados esperados del método.

```
public ResultSet ejecutarConsulta(String consulta, int id) throws SQLException{  17 usages  Fede Calla
    assert consulta != null;
    ResultSet rs = null;
    PreparedStatement stmt = conexion.prepareStatement(consulta);
    if (id != -1) {
    stmt.setInt( parameterIndex: 1, id);
    }
    rs = stmt.executeQuery();


    // stmt.close();
    return rs;
}
```

ejecutarInsert - Test → tabla casos de prueba inserciones

Siguiendo con los testeos de la capa de acceso a la base de datos, testeamos los casos de inserción en la base de datos.

```
public void ejecutarInsert(String insert, AsociadoDTO aDTO)throws SQLException {  13 usages  Fede Calla
    assert   insert != null;
    assert aDTO != null;

    PreparedStatement stmt = conexion.prepareStatement(insert);
    stmt.setString( parameterIndex: 1, aDTO.getNombre());
    stmt.setString( parameterIndex: 2, aDTO.getApellido());
    stmt.setString( parameterIndex: 3, aDTO.getDni());
    stmt.setString( parameterIndex: 4, aDTO.getCalle());
    stmt.setInt( parameterIndex: 5, aDTO.getNumero());
    stmt.setString( parameterIndex: 6, aDTO.getCiudad());
    stmt.setString( parameterIndex: 7, aDTO.getTelefono());
    stmt.executeUpdate();
    stmt.close();
}
```

Nótese que en el testeo de este método, utilizamos como consulta valida el uso de tablas que no existían, esto porque entendimos que la base de datos a un futuro podría crecer y a medida que se agreguen tablas puede crear la confusión de tablas a utilizar, es decir, a regla general, tomamos el concepto de “consulta válida” de la precondición como una consulta *sintácticamente* válida.

ejecutarDelete - Test → tabla casos de prueba eliminaciones

Continuamos testeando la interfaz de eliminación de registros de la base de datos. Distinguimos en este caso que las eliminaciones serán por delete, veremos que es lo que sucede

```
public void ejecutarDelete(String delete, String dni) throws SQLException{  8 usages  Fede Calla
    assert   delete != null;
    assert dni != null;

    PreparedStatement stmt = conexion.prepareStatement(delete);
    stmt.setString( parameterIndex: 1, dni);
    stmt.executeUpdate();
    stmt.close();
}
```

Dentro de los parámetros del método distinguimos los siguientes valores válidos tomando en cuenta las precondiciones del método.

1. dni existente
2. dni inexistente
3. dni vacío
4. dni null

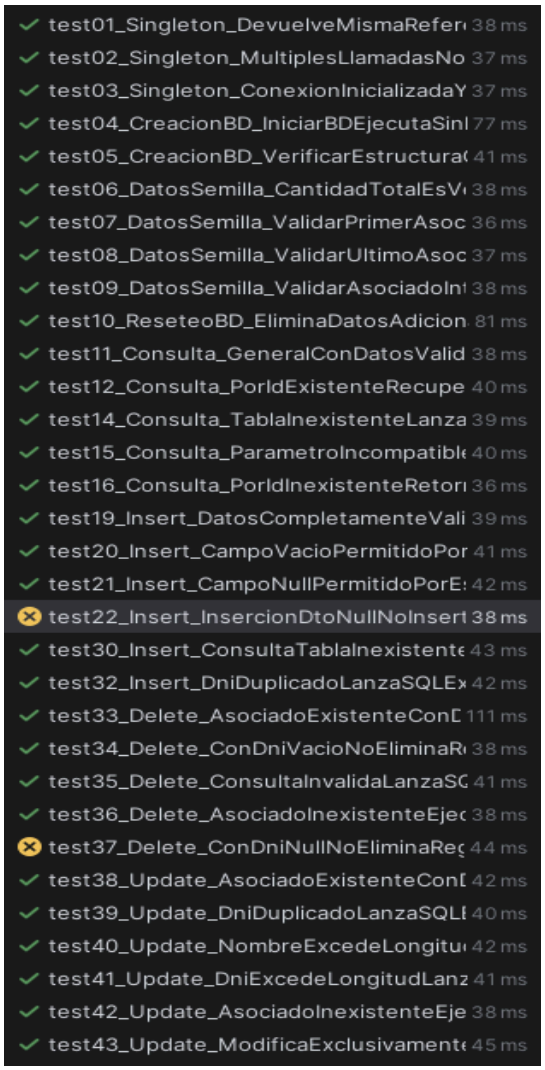
ejecutarActualizacion - Test → tabla casos de prueba actualizaciones

Por último falta crear los casos de prueba para el método ejecutarActualizacion. Igual que con el metodo ejecutarInsert, este metodo utiliza como parámetro AsociadoDTO: aDTO. Partiendo de esta base, crearemos nuestros casos de prueba en base a los posibles valores que puede tomar teniendo en cuenta el contrato.

- aDTO existente en la base de datos
- aDTO con dni existente pero inconsistente con su id de la base de datos
- aDTO con un campo invalido en la base de datos
- aDTO con dni inexistente

```
public void ejecutarActualizacion(String actualizacion, AsociadoDTO aDTO) throws SQLException{  11 usages  Fede Calla
    assert  actualizacion != null;
    assert aDTO != null;
    PreparedStatement stmt = conexion.prepareStatement(actualizacion);
    stmt.setString( parameterIndex: 1, aDTO.getNombre());
    stmt.setString( parameterIndex: 2, aDTO.getApellido());
    stmt.setString( parameterIndex: 3, aDTO.getDni());
    stmt.setString( parameterIndex: 4, aDTO.getCalle());
    stmt.setInt( parameterIndex: 5, aDTO.getNumero());
    stmt.setString( parameterIndex: 6, aDTO.getCiudad());
    stmt.setString( parameterIndex: 7, aDTO.getTelefono());
    stmt.setInt( parameterIndex: 8,  x: 1);
    stmt.executeUpdate();
    stmt.close();
}
```

Análisis de resultados



Una vez planteados los casos de prueba procedemos a correr los test de la clase BaseDeDatosTest para verificar si los resultados de las pruebas son los esperados.

Una vez corridos los testeos obtuvimos 30 pruebas pasadas y 2 errores. Si prestamos atención observamos los siguientes métodos encontraron errores:

- test22_Insert_InsercionDtoNullNoInsertaNada → id T30
- test37_Delete_ConDniNullNoEliminaRegistrosAccidentalmente → id T37

Análisis T30:

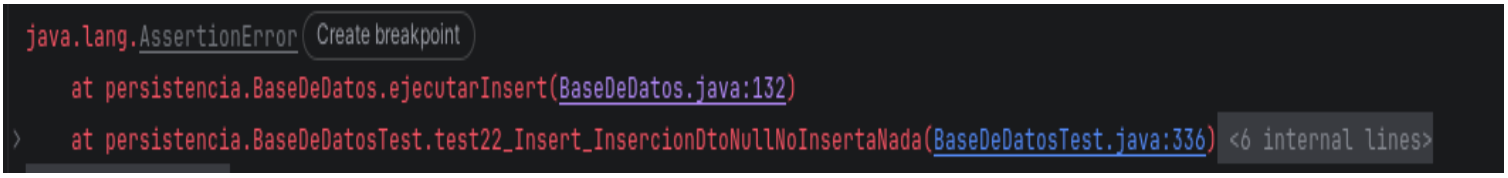
El método “test22_Insert_InsercionDtoNullNoInsertaNada” se encarga de testear el método ejecutarInsert cuando el dto tiene un valor null, el cual entraba dentro de las clases válidas para las pre condiciones especificadas.

```
/**
 * Ejecuta una sentencia INSERT SQL.
 * <b>pre:</b> la sentencia debe ser una sentencia SQL válida.
 * <b>post:</b> se ejecuta la inserción en la base de datos.
 * @param insert La sentencia INSERT SQL a ejecutar.
 * @throws SQLException Si ocurre un error al ejecutar la inserción.
 */
public void ejecutarInsert(String insert, AsociadoDTO aDTO)throws SQLException {  12 usages  Fede Calla
```

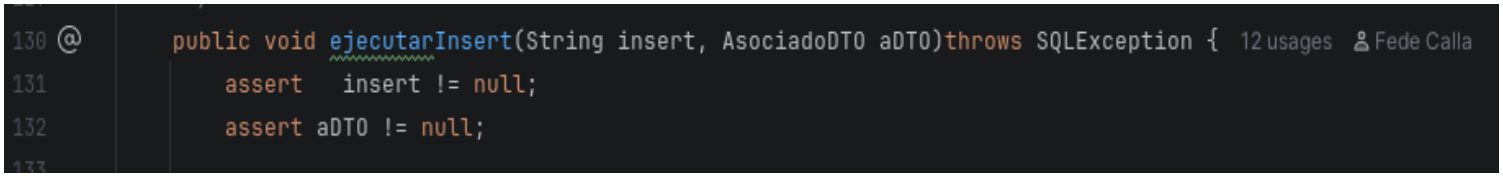
Contrato del método:

– Obsérvese que la única precondición especificada es que la sentencia sea una sentencia válida, lo cual ya explicamos que interpretamos como "válida" – .

Analizando las especificaciones del error encontrado, hallamos que el error fue encontrado



en la línea 132 de la clase BaseDeDatos.



Básicamente el error es un aserto insertado por los desarrolladores del método, de manera tal que corte la ejecución siempre que se pase por parámetro un AsociadoDTO null. Incluso cuando no es realmente un error de ejecución encontrado orgánicamente en la naturaleza de ejecución del método, si es un error forzado por los desarrolladores. Esto igualmente lo consideramos como error ya que el propósito de los asertos son evitar errores de ignorancia de precondiciones en la etapa de desarrollo. No están diseñados totalmente para evitar errores de ejecución. Si este sistema estuviera en producción los asertos serían eliminados e incluso cumpliendo las precondiciones especificadas en el contrato, sería imposible llegar a la postcondición. El uso de un aserto para verificar un caso que no está especificado como precondición excede el propósito de los asertos.
Para arreglar se podrían tomar 3 caminos. El primero y más fácil es especificar como precondición del método que el valor del aDTO.getDni() tendrá que ser distinto del valor null. El siguiente camino sería comprobar con un if el valor de aDTO y no inserte nada en caso de que este sea null. Y el tercero sería la misma validación con un if, pero que si el aDTO resulta ser null devuelve una excepción del tipo IllegalArgumentException().

Análisis T37:

El método “test37_Delete_ConDniNullNoEliminaRegistrosAccidentalmente” se encarga de testear el método ejecutarDelete cuando el dni del Asociado a eliminar toma el valor null. Este valor entra dentro de los valores válidos según el contrato del método

Contrato del Método:

```
/**
 * Ejecuta una sentencia DELETE SQL.
 * <b>pre:</b> la sentencia debe ser una sentencia SQL válida.
 * <b>post:</b> se ejecuta la eliminación en la base de datos.
 * @param delete La sentencia DELETE SQL a ejecutar.
 * @throws SQLException Si ocurre un error al ejecutar la eliminación.
 */
public void ejecutarDelete(String delete, String dni) throws SQLException{  8 usages  Fede Calla
```

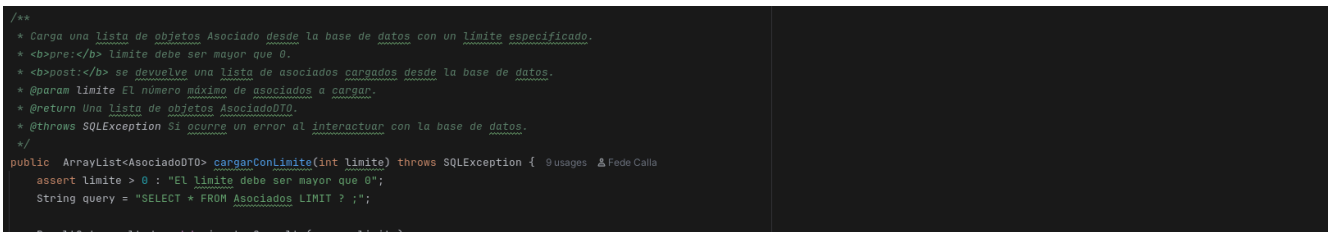
– Obsérvese que la única precondición especificada es que la sentencia sea una sentencia válida, lo cual ya explicamos que interpretamos como “válida” – .

Analizando los detalles de los resultados encontramos que el método ejecutarDelete falla en la línea 154 de la clase BaseDeDatos.

```
java.lang.AssertionError  Create breakpoint
    at persistencia.BaseDeDatos.ejecutarDelete(BaseDeDatos.java:154)
    at persistencia.BaseDeDatosTest.test37_Delete_ConDniNullNoEliminaRegistrosAccidentalmente(BaseDeDatosTest.java:430) <6 internal lines>
```

```
152     public void ejecutarDelete(String delete, String dni) throws SQLException{  8 usages  Fede Calla
153         assert   delete != null;
154         assert dni != null;
```

Devuelta el error no se encontró en un aserto de la clase de testeo, sino que un aserto en la clase a testear. Igual que con el otro error de ejecución, encontramos que los desarrolladores pusieron un aserto como método de validación a un valor de parámetro no especificado en el contrato, o de otra forma, una inconsistencia “contrato - implementación”. De igual manera que con el método ejecutarInsert, entendemos este aserto como un error de implementación, poco sólido y proponemos los mismos 3 caminos posibles para solucionar el error.



Finalmente, el resto de las funciones y casos de prueba planteados devolvieron los resultados esperados. Ahora tocará testear la clase Data Access Object para testear el flujo de trabajos de persistencia desde el lado de la lógica de negocios y no tanto desde el lado de la ejecución de consultas SQL.

Testing Data Access Object

Una vez probada la capa encargada de ejecutar las sentencias SQL (BaseDeDatos), pasamos a DataAccessObject , que es la que usa la lógica de negocio para persistir asociados. A diferencia de BaseDeDatos , esta clase *sí* es responsable del CRUD real de la capa de persistencia: recibe entidades del dominio (Asociado), las convierte a AsociadoDTO y arma las consultas que después ejecuta BaseDeDatos .

El SRS (sección 3.3) especifica lo siguiente: Operación de guardado: el sistema aceptará una entidad de asociado, validará que no sea nula, la convertirá al formato DTO y ejecutará una inserción SQL persistente. Operaciones de carga: el sistema recuperará registros de asociados de manera individual por su identificador único, en lotes acotados con un límite especificado o como una colección completa. Operaciones de actualización y eliminación: el sistema modificará registros existentes o eliminará entradas basándose en criterios únicos como el número de documento (DNI).

Revisando el código de la clase DataAccesObject encontramos los siguientes métodos públicos.

- Public void guardar(Asociado asociado) throws SQLException
- public ArrayLIst<AsociadoDTO> cargarConLimite(int limite) throws SQLException
- public AsociadoDTO cargar(int id) throws SQLException
- public ArrayList<AsociadoDTO> cargarBD() Throws SQLException
- public void eliminar (String dni) Throws SQLException
- public void actualizar(Asociado asociado) throws SQLException
- public AsociadoDTO CrearDTO(Asociado asociado)
- public AsociadoDTO CrearDTO(int id, String nombre, String apellido, String dni, String calle, int numero, String ciudad, String telefono)
- public void cerrarConexion() throws SQLException

Como el DAO depende de BaseDeDatos , del motor MySQL real y de las clases de dominio Asociado y Domicilio , se trata nuevamente de una prueba de integración. Para que cada

prueba sea independiente de las anteriores, antes de cada @Test se ejecuta iniciarBD() , que vuelve a dejar la tabla con los 20 registros semilla. Las verificaciones se hacen consultando la base con JDBC directo (métodos auxiliares contarFilas , buscarPorDni y buscarPorId de la clase UtilBD), para no depender de los mismos métodos del DAO que estamos probando

Escenario	Decisión	Fundamento
1. Base de datos inicializada con los datos semilla	Cubierto	Es el estado normal del sistema después de getInstancia(). Se restablece antes de cada prueba.
2. Conexión cerrada	Cubierto	cerrarConexion() es pública, así que el estado se puede provocar desde afuera (T74)
3. Motor MySQL caído	No testeable	abrirConexion() captura la SQLException y solo imprime la traza, así que la conexión queda en null y la falla aparece recién como NullPointerException en la primera operación. No hay forma de verificar desde la interfaz pública una postcondición sobre ese caso.

La estrategia que adoptamos para el testeo del DAO son las clases de equivalencia, esto debido a que, incluso aunque podamos ver el código, mucho de este bifurca en otras clases con otros métodos y no tanto en bloques de decisión, por lo que resulta más fácil para la detección el armado de las clases de equivalencia.

Los contratos de los métodos del DAO establecen precondiciones explícitas (asociado != null , limite > 0 , id > 0). Con ellas y con el SRS armamos las siguientes clases:

Método	Atributo	Clase Válida	Clase Inválida	Valores límite
guardar	asociado	asociado != null con DNI no registrado	asociado == null / DNI ya registrado	null, DNI "12345678" (existente)

cargarConLimite	límite	límite > 0	límite <= 0	-1, 0, 1, 20 (cantidad de registros), 21
cargar	id	dni != null	dni == null	dni == null dni == 12345678
eliminar	dni	dni existente / dni inexistente	dni == “” / dni == null	"12345678", "00000000", "", null
actualizar	asociado	asociado != null existente	asociado == null	id 1, id 5, null
CrearDTO	asociado / campos	asociado != null, campos válidos	asociado == null, campo null, numero < 0	null, -1

Nótese que en eliminar el Javadoc indica como precondición "id debe ser mayor que 0", aunque el parámetro del método es un String dni . Es una inconsistencia contrato-firma, por lo que para este método tomamos como referencia el mensaje del aserto ("El DNI no puede ser nulo") y

Casos de prueba

Pasaremos ahora a crear los casos de prueba a partir de los módulos especificados, buscando analizar tanto la creación del objeto de acceso a los datos, como el acoplamiento con los demás módulos de la capa de persistencia a partir de operaciones crud.

Constructor y guardado →Tabla de casos de prueba

Para el caso de guardar datos de los asociados, jugamos con los valores de asociado, de manera tal que podemos analizar el comportamiento de la integración con base de datos desde distintos escenarios.

```
/**
 * Constructor de la clase DataAccessObject.
 * <b>post:</b> se crea un objeto DataAccessObject con una instancia de BaseDeDatos.
 */
public DataAccessObject() { bd = BaseDeDatos.getInstancia(); }
```

```java
/**
 * Guarda un objeto Asociado en la base de datos.
 * <b>pre:</b> asociado no debe ser nulo.
 * <b>post:</b> se guarda el asociado en la base de datos.
 * @param asociado El objeto Asociado a guardar.
 * @throws SQLException Si ocurre un error al interactuar con la base de datos.
 */
public void guardar(Asociado asociado) throws SQLException {  8 usages  Fede Calla
    assert asociado != null : "El asociado no puede ser nulo";

    AsociadoDTO aDTO = CrearDTO(asociado);

    String query = "INSERT INTO Asociados(nombre, apellido, DNI, calle, numero, ciudad, telefono) VALUES (?, ?, ?, ?, ?, ?, ?);";

    bd.ejecutarInsert(query, aDTO);
}
```

Carga con límite →[tabla casos de prueba carga con límite]

Los casos de prueba se basaron en el parámetros límite de acuerdo a la implementación del mismo mediante a un resultSet, de esta manera pudimos moldear los distintos escenarios que permitía el contrato del método.

Carga por id → [tabla casos de prueba carga por id]

Siguiendo con el módulo de carga de datos desde la base de datos por id de la base de datos. Como precondición se declara que el id tenga un valor mayor a 0. Y como postcondición se asegura devolver el asociado cargado desde la base de datos. Una excepción SQLException será lanzada si alguna interacción con la base de datos no sale de la manera deseada.

```java
/**
 * Carga un objeto Asociado desde la base de datos por su ID.
 * <b>pre:</b> id debe ser mayor que 0.
 * <b>post:</b> se devuelve el asociado cargado desde la base de datos.
 * @param id El ID del asociado a cargar.
 * @return Un objeto AsociadoDTO.
 * @throws SQLException Si ocurre un error al interactuar con la base de datos.
 */
public AsociadoDTO cargar(int id) throws SQLException {  6 usages  Fede Calla
    assert  id > 0 : "El id debe ser mayor que 0";
    String query = "SELECT * FROM Asociados WHERE id = ? ;";

    ResultSet resultado = bd.ejecutarConsulta(query, id);

    AsociadoDTO aDTO = null;
    aDTO = this.CrearDTO(resultado.getInt( columnLabel: "id"), resultado.getString( columnLabel: "nombre"), resultado.getString( columnLabel: "apellido"), resultado.getString( columnLabel: "dni")
            , resultado.getString( columnLabel: "calle"), resultado.getInt( columnLabel: "numero"), resultado.getString( columnLabel: "ciudad"), resultado.getString( columnLabel: "telefono"));
    resultado.close();
    return aDTO;
}
```

Carga completa → casos de prueba carga completa

Finalmente el último método del módulo de consulta a datos de la base de datos, el cual, no tiene precondiciones, pero asegura devolver la lista de todos los asociados cargados desde la base de datos. Lanzará SQLException si alguna interacción con la base de datos sale mal.

```java
/**
 * Carga todos los objetos Asociado desde la base de datos.
 * <b>post:</b> se devuelve una lista de todos los asociados cargados desde la base de datos.
 * @return Una lista de objetos AsociadoDTO.
 * @throws SQLException Si ocurre un error al interactuar con la base de datos.
 */
public ArrayList<AsociadoDTO> cargarBD() throws SQLException {  6 usages  Fede Calla
    String query = "SELECT * FROM Asociados ;";

    ResultSet resultado = bd.ejecutarConsulta(query,  id: -1);
    ArrayList<AsociadoDTO> asociadosDTO = new ArrayList<>();
    while (resultado.next()) {
        AsociadoDTO aDTO = this.CrearDTO(resultado.getInt( columnLabel: "id"), resultado.getString( columnLabel: "nombre"), resultado.getString( columnLabel: "apellido"),
                resultado.getString( columnLabel: "dni"), resultado.getString( columnLabel: "calle"), resultado.getInt( columnLabel: "numero"),
                resultado.getString( columnLabel: "ciudad"), resultado.getString( columnLabel: "telefono"));
        asociadosDTO.add(aDTO);
    }
    resultado.close();
    return asociadosDTO;
}
```

Eliminación → casos de prueba eliminación

Pasando al lado de las operaciones de eliminación, podemos observar tanto asertos como interacciones acopladas con otros objetos. Tomamos esto como base para el diseño de nuestros casos de prueba.
Como precondición se especifica un id mayor a 0 y se promete como precondición la eliminación del asociado con el dni especificado. También si alguna interacción con la base de datos sale mal, se lanzará la excepción SQLException.

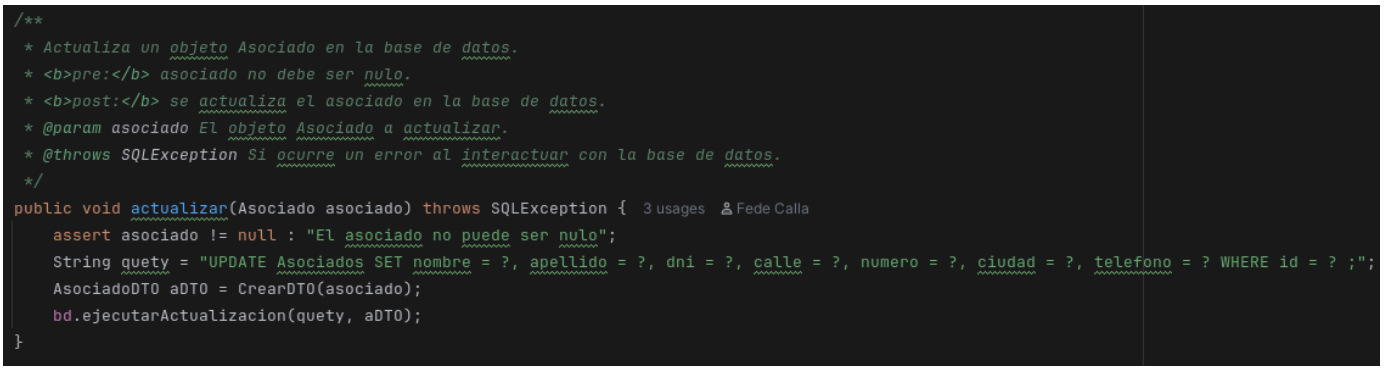
```java
/**
 * Elimina un objeto Asociado de la base de datos por su ID.
 * <b>pre:</b> id debe ser mayor que 0.
 * <b>post:</b> se elimina el asociado de la base de datos.
 * @throws SQLException Si ocurre un error al interactuar con la base de datos.
 */
public void eliminar(String dni) throws SQLException{  8 usages  Fede Calla
    assert !Objects.equals(dni,  b: "") : "El DNI no puede ser nulo";
    String query = "DELETE FROM Asociados WHERE dni = ? ;";

    bd.ejecutarDelete(query, dni);

}
```

Actualización → casos de prueba actualización

Como último método de las operaciones crud, tenemos al método actualizar que especifica de precondicion un asociado no nulo y promente de poscondición el asociado actualizado en la base de datos.
También especifica una excepción en caso de que la interacción con la base de datos de falla.



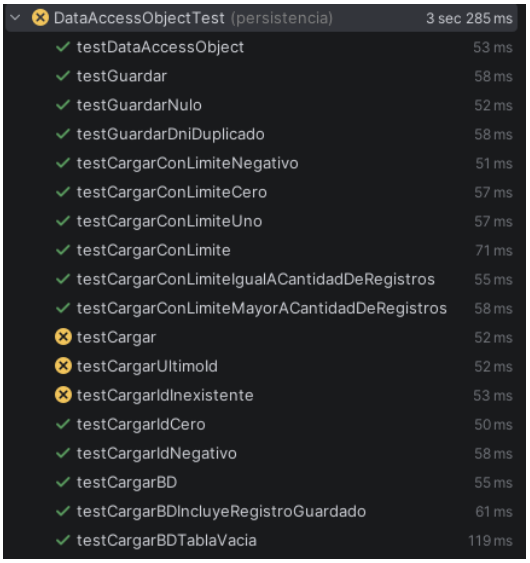
```java
/**
 * Actualiza un objeto Asociado en la base de datos.
 * <b>pre:</b> asociado no debe ser nulo.
 * <b>post:</b> se actualiza el asociado en la base de datos.
 * @param asociado El objeto Asociado a actualizar.
 * @throws SQLException Si ocurre un error al interactuar con la base de datos.
 */
public void actualizar(Asociado asociado) throws SQLException {  3 usages  Fede Calla
    assert asociado != null : "El asociado no puede ser nulo";
    String quety = "UPDATE Asociados SET nombre = ?, apellido = ?, dni = ?, calle = ?, numero = ?, ciudad = ?, telefono = ? WHERE id = ? ;";
    AsociadoDTO aDTO = CrearDTO(asociado);
    bd.ejecutarActualizacion(quety, aDTO);
}
```

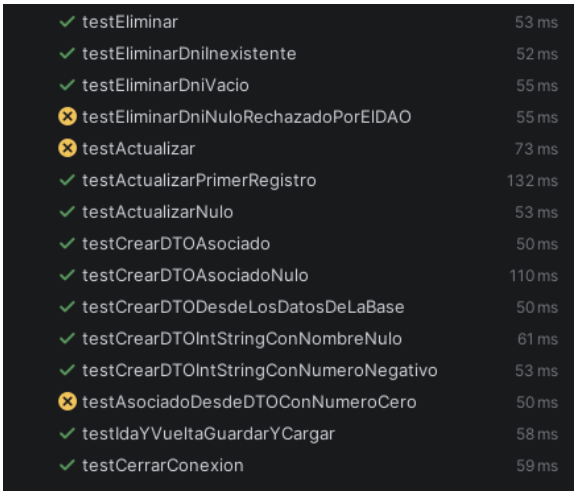
Mapeo Asociado ↔ AsociadoDTO → casos de prueba Mapeo Asociado
Ciclo de vida de la conexión → casos de prueba ciclo de vida conexión

Por último también testeamos 2 módulos los cuales no están especificados en el SRS, sin embargo, nos pareció interesante testearlos porque están estrechamente ligados con las estructura de datos manejadas en el DataAccessObject y el ciclo de vida de la conexión desde el lado de la lógica de negocio realmente.

Análisis de resultados

Una vez planteados los casos de prueba, corrimos DataAccessObjectTest contra MySQL 8.4.





DataAccessObjectTest ejecutó 33 pruebas: 27 pasaron y 6 fallaron. Los métodos que encontraron errores son los siguientes.

- testCargar → id T52
- testCargarUltimoId → id T53
- testCargarIdInexistente → id T54
- testEliminarDniNuloRechazadoPorElDAO → id T63
- testActualizar → id T64
- testAsociadoDesdeDTOConNumeroCero → id T72

<u>Análisis T52, T53 y T54:</u>

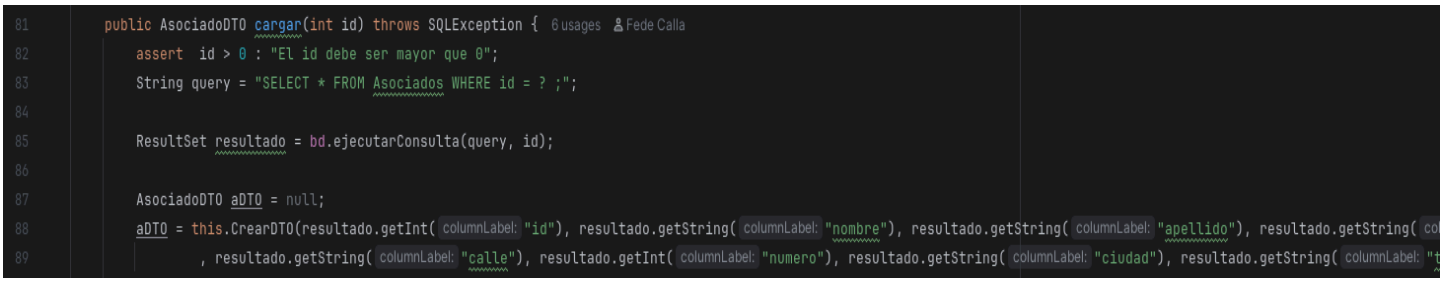
Las tres pruebas invocan cargar(int id) , que según el SRS debe recuperar un asociado de manera individual por su identificador. En los tres casos se lanzó una SQLException

```
java.lang.AssertionError: cargar(1) lanzo SQLException (no posiciona el cursor del ResultSet): Before start of result set
```

```
java.lang.AssertionError: cargar(20) lanzo SQLException: Before start of result set
```

```
java.lang.AssertionError: cargar de un id inexistente deberia devolver null y lanzo: Illegal operation on empty result set.
```

Realizando revisiones de código con el grupo encontramos el error en la línea 88 de la clase DataAccesObject.



```
81    public AsociadoDTO cargar(int id) throws SQLException {  6 usages  Fede Calla
82        assert  id > 0 : "El id debe ser mayor que 0";
83        String query = "SELECT * FROM Asociados WHERE id = ? ;";
84
85        ResultSet resultado = bd.ejecutarConsulta(query, id);
86
87        AsociadoDTO aDTO = null;
88        aDTO = this.CrearDTO(resultado.getInt( columnLabel: "id"), resultado.getString( columnLabel: "nombre"), resultado.getString( columnLabel: "apellido"), resultado.getString( co
89                , resultado.getString( columnLabel: "calle"), resultado.getInt( columnLabel: "numero"), resultado.getString( columnLabel: "ciudad"), resultado.getString( columnLabel: "t
```

En JDBC, el cursor de un ResultSet empieza posicionado antes de la primera fila, y para leer cualquier columna hay que llamar primero a resultado.next() . El método nunca lo hace, por lo que falla para todos los ids sin importar que existan o no. Es decir, la operación de "carga individual" que pide el SRS no funciona en ningún caso. Los otros dos métodos de carga (cargarConLimite y cargarBD) sí llaman a next() dentro de un while y por eso pasaron sus pruebas (T48 a T51 y T57 a T59). Para arreglarlo se podrían tomar dos caminos. El primero es envolver la lectura en un if (resultado.next()) y devolver null cuando el registro no existe, dejándolo especificado como postcondición. El segundo es lanzar una excepción propia (por ejemplo, AsociadoNoEncontradoException) cuando no hay filas, también documentada en el contrato.

Análisis T63:

```
java.lang.AssertionError:
Expected :El DNI no puede ser nulo
Actual   :null
```

La prueba invoca eliminar(null) . La ejecución se corta con un AssertionError.

Analizando el error, y el código del método “eliminar” encontramos dicho método contiene un aserto “assert !Objects.equals(dni, "") : "El DNI no puede ser nulo" el cual verifica que el parámetro DNI no sea la cadena vacía, por lo que un null debería pasar.
La ejecución recién se detiene en el aserto de BaseDeDatos.ejecutarDelete(el mismo que analizamos en T37), y sin mensaje. Además, como ya mencionamos, el javadoc del método indica “pre: id debe ser mayor que 0”, algo que no corresponde con el parámetro que recibe. Este resultado complementa el análisis de T37. Allí concluimos que el aserto de ejecutar Delete no estaba respaldado por el contrato de BaseDeDatos . Acá vemos que la intención

de los desarrolladores sí era rechazar el DNI nulo, pero esa validación quedó en la capa equivocada y mal escrita. La solución es la misma que planteamos antes: corregir el contrato del DAO para que diga "pre: dni no debe ser nulo ni vacío" y reemplazar el aserto por assert dni != null && !dni.isEmpty() , o bien validarlo con un if y lanzar una IllegalArgumentException .
Cabe mencionar, que igualmente, sucede devuelta que hay inconsistencias entre los asertos que hay en los métodos y los contratos especificados en el javadoc, A priori, desde esta prueba de integración desde el DataAccessObject pareciera quedar claras las intenciones de los desarrolladores, sin embargo, me parece pertinente recalcar que tal vez hoy el código y las validaciones funcionen, sin embargo, el día en que el sistema se tuviera que ampliar, refactorizar y tal vez nuevos desarrolladores queden a cargo del codigo, van a saltar estos errores de incongruencias entre los contratos especificados, los asertos insertados y la distribución de responsabilidades entre las distintas capas del módulo de persistencia.

Análisis T64:

La prueba actualiza el teléfono del asociado semilla con id 5 (DNI "33445566") por medio de actualizar(Asociado) . Se lanzó la siguiente excepción:

```
java.lang.AssertionError: actualizar del asociado id=5 lanzo SQLException: Duplicate entry '33445566' for key 'Asociados.DNI'
```

El origen está en la línea 118 de BaseDeDatos.ejecutarActualizacion , que el DAO usa para ejecutar el UPDATE

```
118        stmt.setInt( parameterIndex: 8,  x: 1);
```

El parámetro del WHERE id = ? siempre toma el valor 1, así que toda actualización se aplica sobre el registro con id 1, sin importar qué asociado se quiera modificar. En este caso se intentó escribir el DNI "33445566" en la fila 1, y como ese DNI ya pertenece a la fila 5, MySQL rechazó la operación por la restricción UNIQUE. Si el asociado tuviera un DNI que no está en la tabla, el UPDATE no fallaría y pisaría silenciosamente los datos del registro 1, lo cual es todavía más grave porque no queda ningún rastro del error. En las pruebas de BaseDeDatos (T38 a T41) este comportamiento ya se había observado, pero en ese nivel el método sólo ejecuta la sentencia que recibe y las pruebas se diseñaron sobre el id 1. Recién al probar el DAO, que es el que debe cumplir el SRS ("modificará registros existentes basándose en criterios únicos como el número de documento"), queda claro que la

operación de actualización es incorrecta para 19 de los 20 asociados. T65, que actualiza justamente el asociado con id 1, pasó por coincidencia.

El problema de fondo es que el identificador se pierde en el mapeo: CrearDTO(int id, ...) recibe el id pero lo descarta, porque AsociadoDTO no tiene ese atributo. Para arreglarlo se podrían tomar tres caminos: 1. Usar el criterio único que indica el SRS: cambiar la consulta del DAO a WHERE dni = ? y en ejecutarActualizacion hacer stmt.setString(8, aDTO.getDni()) . 2. Agregar el atributo id a AsociadoDTO y usarlo en el parámetro 8. 3. Agregar un parámetro int id a ejecutarActualizacion

Análisis T72:

La prueba construye un Asociado a partir de un AsociadoDTO con número = 0 , que es un valor válido según el SRS (sección 3.1: "número debe ser mayor o igual a 0"), y que el propio setNumero del DTO acepta después de la corrección que hicimos en la primera parte del trabajo. La construcción falló:

```
java.lang.AssertionError: Un AsociadoDTO valido (numero = 0) no puede convertirse en Asociado: Domicilio exige numero > 0
```

El aserto que corta la ejecución está en la línea 21 de la clase Domicilio (el constructor), y el mismo aserto se repite en la línea 60 (setNumero):

```
19      public Domicilio(String calle, int numero, String ciudad) {  Fede Calla
20          assert calle != null;
21          assert numero > 0;
22          assert ciudad != null;
```

Es la misma inconsistencia que encontramos y corregimos en AsociadoDTO.setNumero() , pero repetida en otra clase. Como consecuencia, un asociado con número 0 puede viajar como DTO, pero la aplicación no puede convertirlo en entidad de dominio. Por ejemplo, Clinica.guardarNuevoAsociado haría new Asociado(datos) y fallaría antes de llegar a la base de datos. La corrección consiste en alinear Domicilio con el SRS (assert numero >= 0) o, si el número 0 no debe ser válido, corregir el SRS y el DTO para que las tres partes coincidan.

Observaciones por inspección del código

- ejecutarConsulta nunca cierra el PreparedStatement (la línea stmt.close() está comentada), por lo que cada consulta deja un recurso abierto en el servidor.
- El registro semilla con id 4 tiene la ciudad ' Mar del Plata' , con un espacio al principio. Es una inconsistencia en los datos iniciales.

Finalmente, el resto de los casos de prueba devolvieron los resultados esperados. Con esto terminamos las pruebas de la capa de persistencia y pasamos a la tercera consigna.

TEST DE INTEGRACIÓN CON MOCKS

Para la tercera parte del trabajo se nos pide probar la clase Ambulancia , la interfaz State y la rutina concurrente RetornoAutomatico . Por un lado, debemos aprovechar el diseño basado en la interfaz State para construir pruebas atómicas con dobles de prueba (mocks). Por otro lado, debemos evaluar el flujo completo de la máquina de estados con los hilos de ejecución, verificando la gestión de recursos compartidos y los bloques sincronizados.
A continuación definiremos un plan por pasos:

- Entender el diseño de la máquina de estados y construir una tabla de transiciones a partir de los contratos.
- Probar la Ambulancia aislada de sus estados, reemplazando State por un mock.
- Probar cada estado concreto aislado de la Ambulancia , reemplazando la Ambulancia por un mock.
- Probar el flujo completo con las clases reales y con varios hilos a la vez.

Para los mocks utilizamos Mockito 4.11, agregado al pom.xml con alcance de test.

Diseño de la máquina de estados

Ambulancia es el contexto del patrón State. Guarda el estado actual (estadoActual), una bandera ocupado que funciona como recurso compartido entre los hilos y una bandera isSimulacionActiva . Extiende de Observable para notificar a la vista mediante objetos NotificacionSimulacion , cuyos tipos son "OK", "NO_AMBULANCIA", "INFO" y "AMBULANCIA".
Los métodos que usan los hilos (atenderDomicilio , trasladarALaClinica , solicitarMantenimiento) son synchronized y siguen el mismo esquema:

1. Mientras la ambulancia está ocupada y la simulación sigue activa, el hilo espera con wait() .
2. Si la simulación terminó mientras esperaba, sale sin hacer nada.
3. Marca la ambulancia como ocupada, delega en el estado actual (estadoActual.SolicitudDe...()) y notifica "OK".

Los métodos que liberan la ambulancia (regresarSinPaciente , volviendoDelTaller) delegan en el estado, ponen ocupado = false y despiertan a los hilos con notifyAll() .
La interfaz State define cuatro eventos: SolicitudDeTraslado , SolicitudDeAtencionDomicilio , SolicitudMantenimiento y RetornoClinica . Hay seis implementaciones concretas. A partir del Javadoc de cada una armamos la **tabla de transiciones** que usamos como oráculo.

El Javadoc de TrasladandoPacienteClinicaState.RetornoClinica dice "Cambia el estado de la ambulancia a DisponibleState". También aceptamos RegresandoClinicaSinPaciente , porque es el estado que usa el resto del diseño después de un servicio.
RetornoAutomatico es un Runnable que, según su Javadoc, "periódicamente intenta retornar la ambulancia a la clínica mientras la simulación está activa", cada 10 segundos (INTERVALO_RETORNO).

Escenarios

Escenario	Decisión	Fundamento
1. Ambulancia con estado simulado (mock de State)	Cubierto	Permite verificar que la Ambulancia delega correctamente y administra el recurso ocupado sin depender de ningún estado concreto.
2. Estado concreto con Ambulancia simulada (mock de Ambulancia)	Cubierto	Permite verificar cada celda de la tabla de transiciones de forma atómica: se captura con un ArgumentCaptor el estado que se pasa a setState() .
3. RetornoAutomatico con estado simulado	Cubierto	Permite verificar si el hilo interactúa con la ambulancia sin esperar cambios de estado reales.

4. Flujo completo con estados reales	Cubierto	Verifica que la composición de las clases respete la tabla de transiciones.
5. Varios hilos compitiendo por la ambulancia	Cubierto	Verifica exclusión mutua, ausencia de deadlocks y que el fin de la simulación libere a los hilos que esperan.
6. Ejecución completa de Asociados y Operario	No testeable de forma determinista	Sus métodos run() usan TiempoMuerto.esperar() (esperas aleatorias de hasta 8 s) y Math.random() para elegir el servicio. En su lugar, los tests crean sus propios hilos que llaman directamente a los métodos de la Ambulancia.

Casos de prueba con mocks (AmbulanciaMockTest)

En todos los casos se usa una Ambulancia real, un Asociado y un Operario reales (sin lanzar sus hilos) y un Observer de prueba que guarda las notificaciones emitidas.
A partir de lo anterior, creamos los siguientes casos de prueba

Ambulancia real + State simulado: delegación → **casos de prueba**

Ambulancia real + State simulado: recurso compartido y sincronización → **casos de prueba**

Estados concretos reales + Ambulancia simulada: transiciones atómicas → casos de prueba

> En cada caso se crea el estado con mock(Ambulancia.class) , se dispara el evento y se verifica con un ArgumentCaptor el estado que se le pasó a setState() . Si el evento debe ser rechazado, se verifica verify(amb, never()).setState(any(State.class)) .

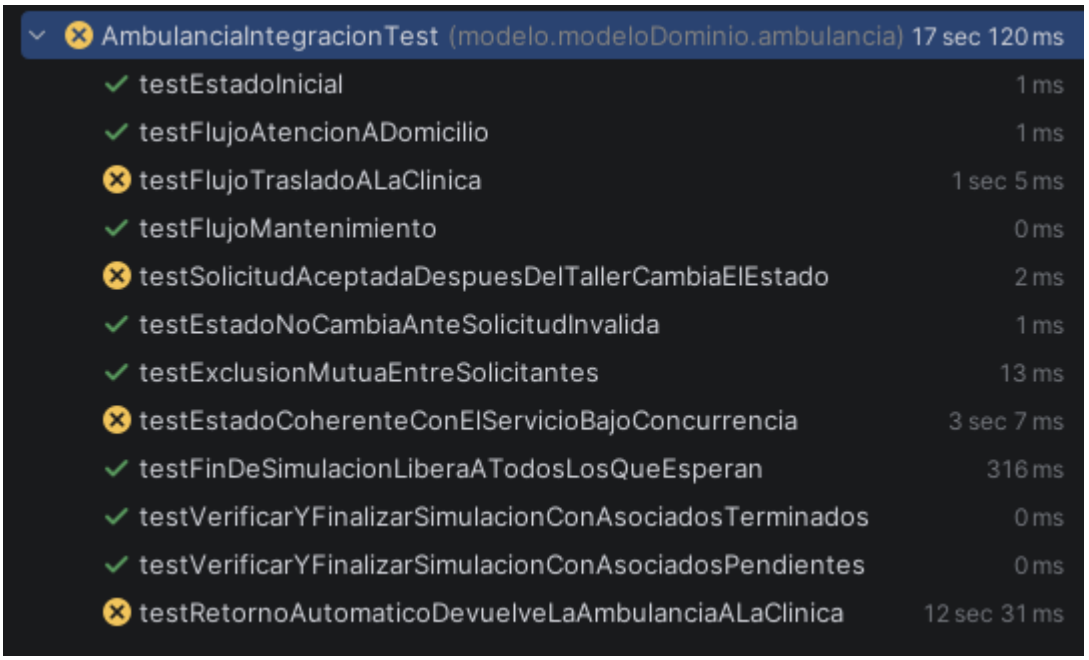
RetornoAutomatico + State simulado → casos de prueba

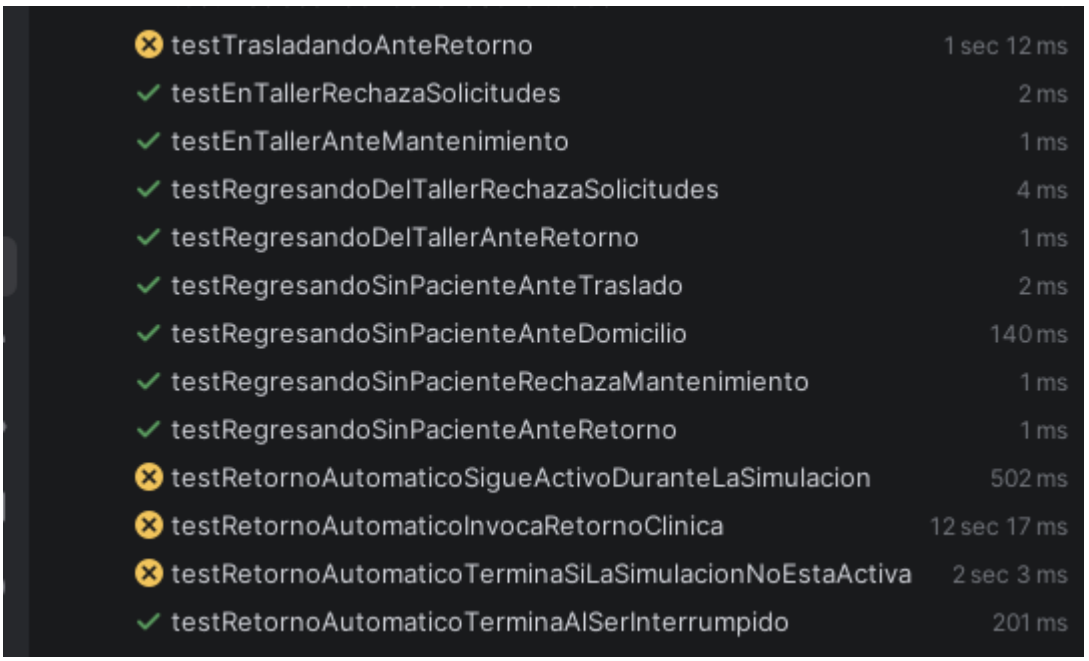
Casos de prueba de integración y concurrencia (AmbulanciaIntegracionTest) → casos de prueba

En esta suite no hay dobles de prueba: se usan la Ambulancia , los seis estados reales y RetornoAutomatico . Para que las pruebas sean rápidas y reproducibles, los hilos se crean dentro del test y llaman directamente a los métodos de la ambulancia, sin TiempoMuerto . Cada join() tiene un tiempo máximo: si un hilo no termina en ese tiempo, lo interpretamos como un deadlock o como una espera sin notificación.
En T121, cada hilo incrementa un contador atómico (AtomicInteger) al obtener la ambulancia y lo decrementa antes de liberarla, y guardamos el valor máximo alcanzado. Si los bloques synchronized y el esquema wait() / notifyAll() fallaran, en algún momento dos hilos estarían "en servicio" al mismo tiempo y el máximo sería mayor a 1.
En T122, mientras un hilo tiene la ambulancia ocupada, ningún otro hilo puede cambiar su estado. Por eso, justo después de que se acepta un servicio, el estado tiene que coincidir con ese servicio. Cada vez que no coincide, guardamos la discrepancia.

Análisis de resultados

Una vez planteados los casos de prueba, corrimos ambas suites.





Entre las dos suites se ejecutaron 46 pruebas: 38 pasaron y 8 fallaron.
Los métodos que encontraron errores son los siguientes:

- testTrasladandoAnteRetorno → id T102
- testRetornoAutomaticoSigueActivoDuranteLaSimulacion → id T111
- testRetornoAutomaticoInvocaRetornoClinica → id T112
- testRetornoAutomaticoTerminaSiLaSimulacionNoEstaActiva → id T113
- testFlujoTrasladoALaClinica → id T117
- testSolicitudAceptadaDespuesDelTallerCambiaElEstado → id T119
- testEstadoCoherenteConElServicioBajoConcurrencia → id T122
- testRetornoAutomaticoDevuelveLaAmbulanciaALaClinica → id T126

Análisis T111, T112, T113 y T126:

Las cuatro pruebas involucran a RetornoAutomatico . Los mensajes obtenidos fueron:

```
T111: El hilo de retorno automatico termino con la simulacion todavia activa
T112: Wanted but not invoked: state.RetornoClinica(); Actually, there were zero interactions with this mock.
T113: Con la simulacion finalizada el hilo no debe seguir ejecutandose
T126: El retorno automatico no cambio el estado; quedo en AtendiendoEnDomicilioState
```

El mock de T112 deja en claro que el hilo nunca interactuó con la ambulancia. El error está en la línea 25 de la clase RetornoAutomatico :

```
25          while (!ambulancia.isSimulacionActiva()) {
26              try {
```

La condición del ciclo está invertida. Con la simulación activa, que es el caso normal, la condición es falsa y run() termina inmediatamente sin hacer ningún retorno, por eso fallan T111, T112 y T126. Con la simulación finalizada ocurre lo contrario: el hilo entra al ciclo, duerme 10 segundos y recién después sale por el break , por eso falla T113. Es exactamente lo opuesto a lo que indica su Javadoc ("mientras la simulación esté activa"). En la aplicación real esto significa que el Thread de RetornoAutomatico que lanza Clinica.iniciarSimulacion termina apenas arranca, y la ambulancia nunca vuelve a la clínica de forma automática. La corrección es quitar la negación: while (ambulancia.isSimulacionActiva()) .

Análisis T102 y T117:

T102 prueba el estado TrasladandoPacienteClinicaState aislado (con la ambulancia simulada), y T117 prueba el flujo completo de traslado con los estados reales. Ambas fallaron por la misma razón:

```
T102: Tras dejar al paciente la ambulancia no vuelve del taller; paso a RegresandoDelTallerState
T117: Tras el traslado la ambulancia quedo en RegresandoDelTallerState
```

El error está en la línea 56 de TrasladandoPacienteClinicaState :

```
55              // this.ambulancia.doNotifyObservers(new NotificacionSimulacion("La
56              this.ambulancia.setState(new RegresandoDelTallerState(this.ambulanc
```

Después de trasladar a un paciente, la ambulancia pasa al estado "regresando del taller", aunque nunca estuvo en el taller. El Javadoc del mismo método dice "Cambia el estado de la ambulancia a DisponibleState", así que es una inconsistencia contrato-implementación.

Además, en la línea 54 el método ejecuta Thread.sleep(1000) . Como RetornoClinica() se invoca desde métodos synchronized de Ambulancia (regresarSinPaciente y retornoAutomatico), el hilo duerme un segundo **mientras tiene el lock de la ambulancia**, y todos los demás hilos quedan bloqueados durante ese tiempo. El catch (Exception e) vacío también descarta una posible interrupción del hilo. Lo notamos durante la ejecución, porque cada traslado demoraba un segundo más que los demás servicios.

La corrección es cambiar la transición a new DisponibleState(...) (o a RegresandoClinicaSinPacienteState , para mantener la simetría con la atención a domicilio) y quitar el sleep de dentro del bloque sincronizado.

Análisis T119:

Esta es la falla más interesante del trabajo, porque **no la detecta ninguna prueba con mocks:** cada clase, probada por separado, cumple su propio contrato. Solo aparece al combinar las clases reales. El mensaje fue:

```
La solicitud se acepto pero el estado quedo en RegresandoDelTallerState: la atencion se pierde
```

La secuencia es la siguiente:

1. El operario solicita mantenimiento: Disponible → EnTaller, ocupado = true .
2. La ambulancia vuelve del taller (Ambulancia.volviendoDelTaller , líneas 154 a 157): EnTaller → RegresandoDelTaller y ocupado = false .
3. Un asociado pide atención a domicilio. Como ocupado == false , la ambulancia no lo hace esperar: pone ocupado = true , delega en el estado y notifica "OK".
4. Pero RegresandoDelTallerState rechaza la solicitud (T105, comportamiento correcto según su contrato), así que el estado no cambia.

El resultado es que el asociado recibe una notificación de que fue atendido, pero la ambulancia nunca pasó a atenderlo: **la solicitud se pierde**. El problema es que la bandera ocupado y el estado actual se contradicen: la ambulancia figura como libre mientras está en un estado que no acepta pedidos. El único que puede sacarla de ese estado es RetornoAutomatico , que como vimos en T111 no funciona.

Para arreglarlo se podrían tomar dos caminos. El primero es que volviendoDelTaller complete la vuelta y deje la ambulancia en Disponible (invocando también estadoActual.RetornoClinica()) antes de liberarla. El segundo es mantener ocupado = true hasta que el retorno automático la deje en Disponible. En cualquiera de los dos casos, la regla a respetar es que **la ambulancia sólo puede figurar libre en un estado que acepte solicitudes**.

Análisis T122:

Esta prueba de concurrencia combina los dos errores anteriores. Los dos hilos alternan atención a domicilio y traslado, y se registraron tres discrepancias:

```
Servicios aceptados con un estado incoherente: [domicilio -> RegresandoDelTallerState, domicilio -> RegresandoDelTallerState,
domicilio -> RegresandoDelTallerState]
```

Después de cada traslado la ambulancia queda en RegresandoDelTallerState (error de T102), y el siguiente pedido de atención a domicilio se acepta pero se pierde (error de T119). Con los hilos de la simulación real, que eligen el servicio al azar, esto ocurre en aproximadamente la mitad de los servicios posteriores a un traslado.
Finalmente, el resto de las pruebas devolvieron los resultados esperados. En particular, T121 y T123 confirman que la sincronización de Ambulancia está bien implementada: la exclusión mutua se respetó en los 100 servicios concurrentes, no hubo deadlocks y el fin de la simulación liberó a todos los hilos que esperaban. Los errores encontrados no son de sincronización, sino **de estado**: transiciones incorrectas y falta de coherencia entre la bandera ocupado y el estado actual.
Para confirmar que las fallas se deben al código y no a las pruebas, aplicamos en una copia del proyecto las correcciones propuestas para RetornoAutomatico , TrasladandoPacienteClinicaState y volviendoDelTaller . Con esas correcciones, las dos suites pasaron 46/46 en tres ejecuciones consecutivas, sin fallas intermitentes.

CONCLUSIONES

A lo largo del trabajo aplicamos tres niveles de prueba, y cada uno encontró errores que los otros no podían encontrar.
En las pruebas unitarias de AsociadoDTO , la suite detectó de forma aislada y sin servicios externos la inconsistencia de setNumero() con el SRS. Luego, en las pruebas de integración, apareció la misma inconsistencia en Domicilio (T72). Esto muestra que corregir una clase no alcanza si la misma regla está duplicada en otras partes del sistema.
En las pruebas de integración con la base de datos, comprobamos que el Singleton, la inicialización automática y las operaciones con PreparedStatement funcionan, incluyendo la protección contra datos duplicados. Las fallas más graves aparecieron recién al probar el DAO, que es la capa que debe cumplir el SRS: la carga por id no funciona para ningún registro (T52 a T54) y la actualización siempre se aplica sobre el registro 1 (T64). Además, la prueba del flujo completo a través de Clinica pasó sin errores porque no usa esos métodos. Probar solo los flujos de punta a punta hubiera dejado ocultos los dos errores.

En las pruebas con mocks, el diseño desacoplado basado en la interfaz State nos permitió probar la Ambulancia sin sus estados y cada estado sin la Ambulancia . Así, cada falla quedó atribuida a una clase concreta: RetornoAutomatico (condición invertida) y TrasladandoPacienteClinicaState (transición incorrecta). Las pruebas de integración y concurrencia confirmaron que la sincronización es correcta, pero encontraron un error que ningún mock podía detectar: la contradicción entre la bandera ocupado y el estado actual (T119 y T122).

En varios análisis (T30, T37 y T63) encontramos que los asertos se usan para validar condiciones que no figuran en los contratos, o que los contratos no corresponden con la firma del método. Mantener los contratos actualizados es lo que nos permite decidir con fundamento si una falla es un error del código o de la especificación.

Y como conclusión final, nos gustaría recalcar que gran parte de los errores no son errores algorítmicos o de codificación, sino que son errores de inconsistencia entre “lo que se debe hacer” y “lo que realmente se hizo”. Muchas veces estas inconsistencias entre la documentación de lo realizado y lo realmente realizado son los errores silenciosos más peligrosos. Las posibles complicaciones en equipos de trabajo más grandes o la escalabilidad de módulos del sistema es lo que sacan a la luz estos errores y los hace muchísimo más difíciles de encontrar. Es por eso que nos pareció una apreciación interesante el hecho de mencionar como fuente de los errores la documentación(en este caso en los javadoc).